

Laboratorio 3 - Mi despliegue CI/CD en Kubernetes

El desafío final consiste en llevar una aplicación web desde el código fuente hasta un despliegue funcional en Kubernetes, usando Docker, registros de imágenes, Jenkins, Jenkinsfile, credenciales y manifiestos Kubernetes.

La tarea es individual. No basta con copiar los ejemplos de clases: debes cambiar nombres, imágenes, variables y configuraciones para demostrar que entiendes cómo se conectan las piezas.

Objetivo del desafío

Debes construir un flujo completo que permita:

- Crear una imagen Docker propia para la aplicación entregada.
- Publicar la imagen en Docker Hub o GitHub Container Registry.
- Desplegar la aplicación en un cluster Kubernetes local.
- Automatizar el proceso con Jenkins usando agentes kubernetes.
- Demostrar que la aplicación funciona usando kubectl y curl.

Personalización obligatoria

Cada alumno debe usar nombres propios. Si te llamas Juan Perez, puedes usar el formato juan-perez.

Ejemplo:

- Namespace: ns-juan-perez
- Deployment: app-juan-perez
- Service: svc-juan-perez
- ConfigMap: config-juan-perez
- Secret: secret-juan-perez
- Imagen Docker: usuario/tarea-final:juan-perez
- Imagen Docker: usuario/tarea-final:\${APP_VERSION}
- APP_VERSION: 3.0.0
- Pipeline Jenkins: Jenkinsfile.Juan-Perez

Requisitos

La entrega debe cumplir con lo siguiente:

1. Usar un cluster local: Docker Desktop Kubernetes, kubeadm, Minikube, Kind, k3d u otro.
2. Crear un Dockerfile funcional y publicar la imagen en el registry de dockerhub y github
3. Crear Namespace, Deployment, Service, ConfigMap y Secret.
4. El Deployment debe usar la imagen publicada según tag de nombre (no versión) y tener al menos 2 réplicas. Puedes usar la de dockerhub o github, tu eliges.
5. La aplicación debe leer una variable desde un ConfigMap y un valor desde un Secret. Para el caso del configmap, debes guardar y leer la variable **AMBIENTE**, y en el caso del secreto la variable **API_KEY**
6. El Jenkinsfile debe tener stages: install, test, build, push y deploy.
7. El pipeline debe escribirse usando un agente de tipo kubernetes y escribiendo un agent.yaml
8. Las credenciales no deben quedar escritas directamente en el Jenkinsfile.

Entregables

Debes una carpeta comprimida con:

- Dockerfile
- .dockerignore
- Jenkinsfile
- Archivo entrega.yaml con los manifiestos Kubernetes
- Archivo agent.yaml con configuración de agente kubernetes.
- Log de pipeline de jenkins
- README.md con instrucciones de ejecución o instrucciones generales.
- Carpeta evidencias/ con capturas o salidas de comandos

Evidencias obligatorias

Incluye evidencia de estos comandos en tu terminal:

- `kubectl cluster-info`
- `kubectl get nodes`
- `kubectl get pods -n ns-nombre-apellido`
- `kubectl get deployment -n app-nombre-apellido`
- `kubectl get svc -n svc-nombre-apellido`
- `kubectl logs deployment/app-nombre-apellido -n ns-nombre-apellido`
- `kubectl exec deployment/app-nombre-apellido -n ns-nombre-apellido -- printenv`

- `kubectl get configmap config-nombre-apellido`
- `kubectl get secret secret-nombre-apellido`

También debes incluir evidencia del pipeline Jenkins ejecutado correctamente y una prueba de consulta a la aplicación con:

- `kubectl port-forward svc/svc-nombre-apellido 8080:80 -n ns-nombre-apellido`
- `curl http://localhost:8080/lab`

Tips

- Válida manualmente `docker build`, `docker push` y `kubectl apply` antes de automatizar en Jenkins.
- Revisa que los labels del Deployment coincidan con el selector del Service.
- Si tu pod no inicia, usa `kubectl describe pod` y `kubectl logs` antes de modificar al azar.